Assignment-6

P KARTIKEYA RAJ

2023001492

Repo link

1. Write a program to implement TIC – TAC - TOE game.

```
import tkinter as tk
from tkinter import messagebox
import random
import math

class TicTacToe:
    def __init__(self, root):
        self.root = root
        self.root.title("Advanced Tic Tac Toe - AI")

        self.board = [" "] * 9
        self.human = "X"
        self.ai = "O"
        self.current_turn = "X"

        self.difficulty = "Hard"

        self.human_score = 0
        self.ai_score = 0
        self.draw_score = 0

        self.create_ui()

    # ================ UI ================

    def create_ui(self):

        top_frame = tk.Frame(self.root)
        top_frame.pack()
```

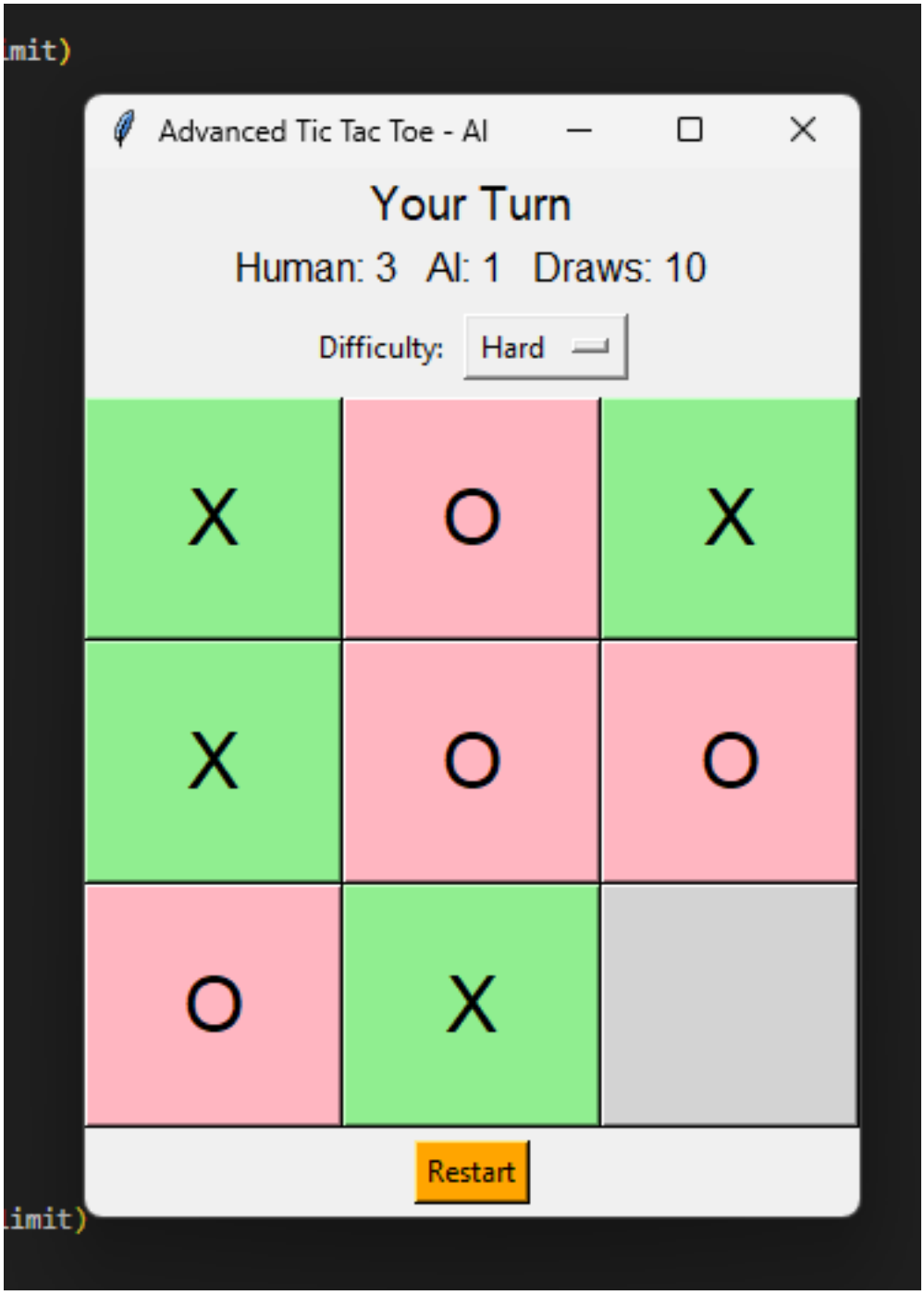
```
        self.status_label = tk.Label(top_frame,
text="Your Turn",
                                     font=("Arial",
14))
        self.status_label.pack()

        self.score_label = tk.Label(top_frame,
                                    text="Human:
0   AI: 0   Draws: 0",
                                    font=("Arial",
12))
        self.score_label.pack()

        level_frame = tk.Frame(self.root)
        level_frame.pack(pady=5)

        tk.Label(level_frame, text="Difficulty:
").pack(side=tk.LEFT)

        self.level_var = tk.StringVar(value="Hard")

        tk.OptionMenu(level_frame, self.level_var,
                      "Easy", "Medium",
"Hard").pack(side=tk.LEFT)

        self.buttons = []
        board_frame = tk.Frame(self.root)
        board_frame.pack()

        for i in range(9):
            btn = tk.Button(board_frame,
                            text=" ",
                            font=("Arial", 24),
                            width=5,
                            height=2,
                            bg="lightgray",
                            command=lambda i=i:
self.player_move(i))
            btn.grid(row=i // 3, column=i % 3)
            self.buttons.append(btn)

        tk.Button(self.root, text="Restart",
                  command=self.reset_game,
                  bg="orange").pack(pady=5)
```

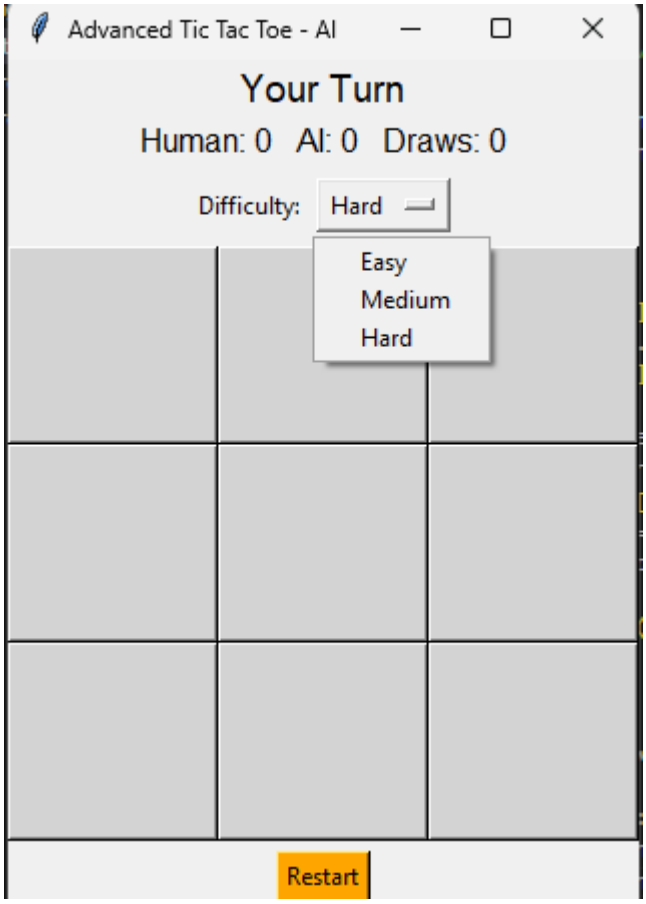
```
    # ================ GAME LOGIC ================

    def player_move(self, index):
        if self.board[index] == " " and
self.current_turn == self.human:
            self.make_move(index, self.human)

            if not self.check_game_over():
                self.current_turn = self.ai
                self.root.after(400, self.ai_move)

    def ai_move(self):
        self.difficulty = self.level_var.get()

        if self.difficulty == "Easy":
            move = self.random_move()

        elif self.difficulty == "Medium":
            if random.random() < 0.4:
                move = self.random_move()
            else:
                move = self.best_move(depth_limit=3)

        else:  # Hard
            move = self.best_move(depth_limit=None)

        self.make_move(move, self.ai)

        if not self.check_game_over():
            self.current_turn = self.human
            self.status_label.config(text="Your
Turn")

    def make_move(self, index, player):
        self.board[index] = player
        self.buttons[index].config(text=player,
                                   bg="#90EE90" if
player == "X" else "#FFB6C1")

    def random_move(self):
        empty = [i for i in range(9) if self.board[i]
== " "]
        return random.choice(empty)
```

```
    # ================ MINIMAX ================

    def best_move(self, depth_limit):
        best_score = -math.inf
        moves = []

        for i in range(9):
            if self.board[i] == " ":
                self.board[i] = self.ai
                score = self.minimax(0, False, -
math.inf, math.inf, depth_limit)
                self.board[i] = " "

                if score > best_score:
                    best_score = score
                    moves = [i]
                elif score == best_score:
                    moves.append(i)

        return random.choice(moves)

    def minimax(self, depth, is_max, alpha, beta,
depth_limit):

        winner = self.check_winner()

        if winner == self.ai:
            return 10 - depth
        elif winner == self.human:
            return depth - 10
        elif " " not in self.board:
            return 0

        if depth_limit is not None and depth >=
depth_limit:
            return 0

        if is_max:
            max_eval = -math.inf
            for i in range(9):
                if self.board[i] == " ":
                    self.board[i] = self.ai
```

```
                    eval = self.minimax(depth+1,
False, alpha, beta, depth_limit)
                    self.board[i] = " "
                    max_eval = max(max_eval, eval)
                    alpha = max(alpha, eval)
                    if beta <= alpha:
                        break
            return max_eval
        else:
            min_eval = math.inf
            for i in range(9):
                if self.board[i] == " ":
                    self.board[i] = self.human
                    eval = self.minimax(depth+1,
True, alpha, beta, depth_limit)
                    self.board[i] = " "
                    min_eval = min(min_eval, eval)
                    beta = min(beta, eval)
                    if beta <= alpha:
                        break
            return min_eval

    # ================ WIN CHECK ================

    def check_winner(self):
        combos = [
            [0,1,2],[3,4,5],[6,7,8],
            [0,3,6],[1,4,7],[2,5,8],
            [0,4,8],[2,4,6]
        ]
        for combo in combos:
            if self.board[combo[0]] != " " and \
               self.board[combo[0]] ==
self.board[combo[1]] == self.board[combo[2]]:
                return self.board[combo[0]]
        return None

    def check_game_over(self):
        winner = self.check_winner()

        if winner:
            if winner == self.human:
                self.human_score += 1
```

```
                messagebox.showinfo("Game Over", "You
Win!")
            else:
                self.ai_score += 1
                messagebox.showinfo("Game Over", "AI
Wins!")

            self.update_score()
            self.reset_game()
            return True

        elif " " not in self.board:
            self.draw_score += 1
            messagebox.showinfo("Game Over", "Draw!")
            self.update_score()
            self.reset_game()
            return True

        return False

    def update_score(self):
        self.score_label.config(
            text=f"Human: {self.human_score}   AI:
{self.ai_score}   Draws: {self.draw_score}"
        )

    def reset_game(self):
        self.board = [" "] * 9
        self.current_turn = self.human
        self.status_label.config(text="Your Turn")
        for btn in self.buttons:
            btn.config(text=" ", bg="lightgray")

# ================ MAIN ================

root = tk.Tk()
game = TicTacToe(root)
root.mainloop()
```


Advanced Tic Tac Toe - AI
Your Turn
Human: 3 AI: 1 Draws: 10
Difficulty: Hard
X O X
X O O
O X
Restart


Advanced Tic Tac Toe - AI
Your Turn
Human: 0 AI: 0 Draws: 0
Difficulty:
Hard
Easy
Medium
Hard
Restart

2. Write a program to implement Hangman game (Or Wordle).

```
import tkinter as tk
from tkinter import messagebox
import random
import time

class Hangman:
    def __init__(self, root):
        self.root = root
        self.root.title("HANGMAN")
        self.root.configure(bg="#121213")
        self.root.resizable(False, False)

        self.words = [
            "PYTHON", "HANGMAN", "COMPUTER",
"PROGRAM", "ALGORITHM",
            "VARIABLE", "FUNCTION", "STRING",
"INTEGER", "RANDOM"
        ]

        # Game variables
        self.max_tries = 6
        self.reset_game()

        # UI
        self.create_ui()
```

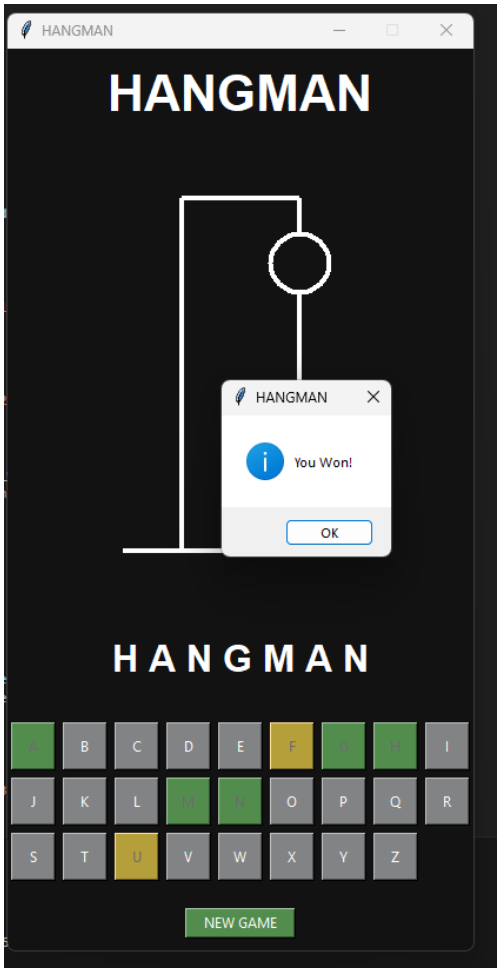
```
    def reset_game(self):
        self.word = random.choice(self.words).upper()
        self.guessed_letters = []
        self.wrong_guesses = 0
        self.display_word = ["_" if c.isalpha() else
c for c in self.word]
        self.game_over = False

    def create_ui(self):
        # Title
        self.title_label = tk.Label(self.root,
text="HANGMAN", font=("Helvetica", 32, "bold"),
                                    fg="white",
bg="#121213")
        self.title_label.pack(pady=10)

        # Canvas for hangman drawing
        self.canvas = tk.Canvas(self.root, width=300,
height=400, bg="#121213", highlightthickness=0)
        self.canvas.pack()

        # Display word
        self.word_label = tk.Label(self.root, text="
".join(self.display_word),
                                   font=("Helvetica",
24, "bold"), fg="white", bg="#121213")
        self.word_label.pack(pady=20)

        # Keyboard
```

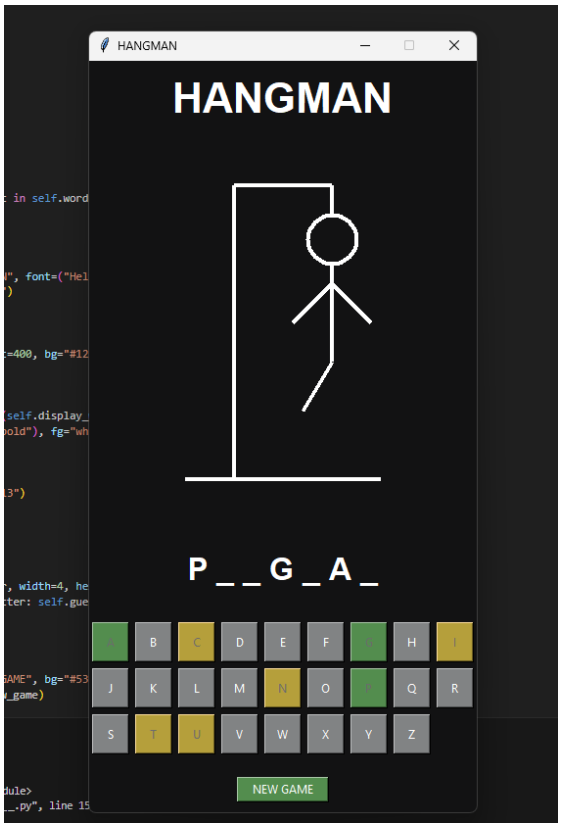
```
        self.keyboard_frame = tk.Frame(self.root,
bg="#121213")
        self.keyboard_frame.pack(pady=10)

        self.keyboard_buttons = {}
        letters = "ABCDEFGHIJKLMNOPQRSTUVWXYZ"
        for i, letter in enumerate(letters):
            btn = tk.Button(self.keyboard_frame,
text=letter, width=4, height=2, bg="#818384",
                            fg="white",
command=lambda l=letter: self.guess_letter(l))
            btn.grid(row=i//9, column=i%9, padx=3,
pady=3)
            self.keyboard_buttons[letter] = btn

        # New game button
        self.new_game_btn = tk.Button(self.root,
text="NEW GAME", bg="#538d4e", fg="white", width=12,
                                      command=self.st
art_new_game)
        self.new_game_btn.pack(pady=10)

        # Draw initial gallows
        self.draw_gallows()

    def start_new_game(self):
        self.reset_game()
        self.word_label.config(text="
".join(self.display_word))
        for btn in self.keyboard_buttons.values():
```

```
            btn.config(state=tk.NORMAL, bg="#818384")
        self.canvas.delete("hangman")
        self.draw_gallows()

    def guess_letter(self, letter):
        if self.game_over:
            return
        self.keyboard_buttons[letter].config(state=tk
.DISABLED)
        if letter in self.word:
            self.keyboard_buttons[letter].config(bg="
#538d4e")  # green for correct
            for i, c in enumerate(self.word):
                if c == letter:
                    self.display_word[i] = letter
            self.word_label.config(text="
".join(self.display_word))
            if "_" not in self.display_word:
                self.game_over = True
                messagebox.showinfo("HANGMAN", "You
Won! ")
        else:
            self.keyboard_buttons[letter].config(bg="
#b59f3b")  # yellow/orange for wrong
            self.wrong_guesses += 1
            self.draw_hangman_part(self.wrong_guesses
)
            if self.wrong_guesses >= self.max_tries:
                self.game_over = True
```

```
                messagebox.showinfo("HANGMAN", f"You
Lost! The word was: {self.word}")


    # Draw gallows
    def draw_gallows(self):
        self.canvas.create_line(50, 350, 250, 350,
fill="white", width=4)  # base
        self.canvas.create_line(100, 350, 100, 50,
fill="white", width=4)  # pole
        self.canvas.create_line(100, 50, 200, 50,
fill="white", width=4)   # top beam
        self.canvas.create_line(200, 50, 200, 80,
fill="white", width=4)   # rope


    # Draw hangman parts with animation
    def draw_hangman_part(self, step):
        # step 1 = head, 2 = body, 3 = left arm, 4 =
right arm, 5 = left leg, 6 = right leg
        if step == 1:
            self.canvas.create_oval(175, 80, 225,
130, outline="white", width=4, tags="hangman")  #
head
        elif step == 2:
            self.canvas.create_line(200, 130, 200,
230, fill="white", width=4, tags="hangman")  # body
        elif step == 3:
            self.canvas.create_line(200, 150, 160,
190, fill="white", width=4, tags="hangman")  # left
arm
        elif step == 4:
```

```
            self.canvas.create_line(200, 150, 240,
190, fill="white", width=4, tags="hangman")  # right
arm
        elif step == 5:
            self.canvas.create_line(200, 230, 170,
280, fill="white", width=4, tags="hangman")  # left
leg
        elif step == 6:
            self.canvas.create_line(200, 230, 230,
280, fill="white", width=4, tags="hangman")  # right
leg
# Main
if __name__ == "__main__":
    root = tk.Tk()
    hangman = Hangman(root)
    root.mainloop()
```


HANGMAN
HANGMAN
HANGMAN
You Won!
OK
H A N G M A N
A B C D E F G H I
J K L M N O P Q R
S T U V W X Y Z
NEW GAME


HANGMAN
HANGMAN
P _ _ G _ A _
A B C D E F G H I
J K L M N O P Q R
S T U V W X Y Z
NEW GAME

WORDLE

```
import tkinter as tk
from tkinter import messagebox
import random

class Wordle:
    def __init__(self, root):
        self.root = root
        self.root.title("WORDLE")
        self.root.configure(bg="#121213")
        self.root.resizable(False, False)

        # Load words
        self.load_words()

        # Stats
        self.wins = 0
        self.streak = 0
        self.games = 0

        # UI
        self.create_ui()
        self.new_game()
```

```
    # ======= Load words =======
    def load_words(self):
        try:
            with open("words.txt", "r") as f:
                text = f.read().upper()
            raw_words = text.replace(",", "
").split()
            self.words = [w.strip() for w in
raw_words if len(w.strip()) == 5 and w.isalpha()]
            if not self.words:
                raise ValueError("No valid words
found")
        except Exception:
            messagebox.showwarning("Warning",
"words.txt missing or invalid. Using default words.")
            self.words = ["APPLE", "GRAPE", "HOUSE",
"PLANT", "BRAIN", "LIGHT"]


    # ======= New game =======
    def new_game(self):
        self.target = random.choice(self.words)
        self.current_row = 0
        self.current_col = 0
        self.game_over = False
        self.board = [["" for _ in range(5)] for _ in
range(6)]
        for row in self.tiles:
            for tile in row:
                tile.config(text="", bg="#121213")
        for key in self.keyboard_buttons.values():
```

```
            key.config(bg="#818384")


    # ======= UI =======
    def create_ui(self):
        title = tk.Label(self.root, text="WORDLE",
font=("Helvetica", 28, "bold"), fg="white",
bg="#121213")
        title.pack(pady=10)


        self.stats_label = tk.Label(self.root,
text="Wins: 0  |  Streak: 0", fg="white",
bg="#121213")
        self.stats_label.pack()


        board_frame = tk.Frame(self.root,
bg="#121213")
        board_frame.pack(pady=10)
        self.tiles = []
        for r in range(6):
            row_tiles = []
            for c in range(5):
                tile = tk.Label(board_frame, text="",
width=4, height=2, font=("Helvetica", 24, "bold"),
                                bg="#121213",
fg="white", relief="solid", bd=2)
                tile.grid(row=r, column=c, padx=5,
pady=5)
                row_tiles.append(tile)
            self.tiles.append(row_tiles)
```

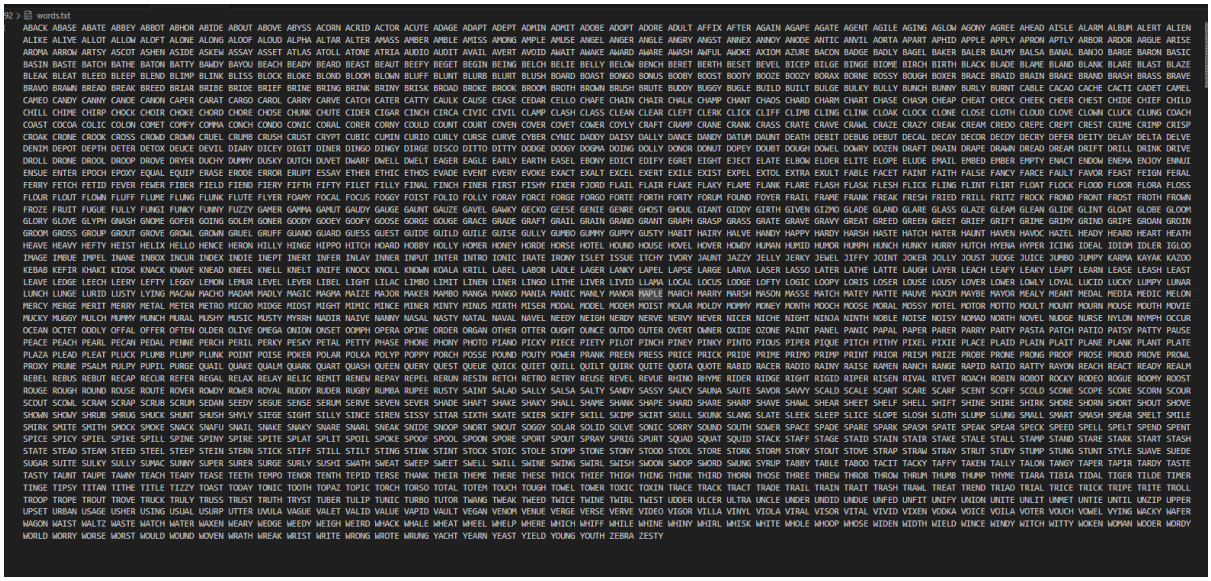
```
        # Keyboard
        keyboard_frame = tk.Frame(self.root,
bg="#121213")
        keyboard_frame.pack()
        self.keyboard_buttons = {}
        rows = ["QWERTYUIOP", "ASDFGHJKL", "ZXCVBNM"]
        for row in rows:
            frame = tk.Frame(keyboard_frame,
bg="#121213")
            frame.pack()
            for letter in row:
                btn = tk.Button(frame, text=letter,
width=4, height=2, bg="#818384", fg="white",
                                command=lambda
l=letter: self.handle_letter(l))
                btn.pack(side=tk.LEFT, padx=3,
pady=3)
                self.keyboard_buttons[letter] = btn


        control_frame = tk.Frame(self.root,
bg="#121213")
        control_frame.pack(pady=5)
        tk.Button(control_frame, text="ENTER",
width=8, bg="#3a3a3c", fg="white",
command=self.submit_guess).pack(side=tk.LEFT, padx=5)
        tk.Button(control_frame, text="DELETE",
width=8, bg="#3a3a3c", fg="white",
command=self.delete_letter).pack(side=tk.LEFT,
padx=5)
        tk.Button(control_frame, text="NEW GAME",
width=10, bg="#538d4e", fg="white",
command=self.new_game).pack(side=tk.LEFT, padx=5)
```

```
        self.root.bind("<Key>", self.key_press)

    # ======= Input handling =======
    def key_press(self, event):
        if self.game_over:
            return
        key = event.char.upper()
        if key.isalpha() and len(key) == 1:
            self.handle_letter(key)
        elif event.keysym == "Return":
            self.submit_guess()
        elif event.keysym == "BackSpace":
            self.delete_letter()

    def handle_letter(self, letter):
        if self.current_col < 5 and not
self.game_over:
            self.board[self.current_row][self.current
_col] = letter
            self.tiles[self.current_row][self.current
_col].config(text=letter)
            self.current_col += 1

    def delete_letter(self):
        if self.current_col > 0 and not
self.game_over:
            self.current_col -= 1
```

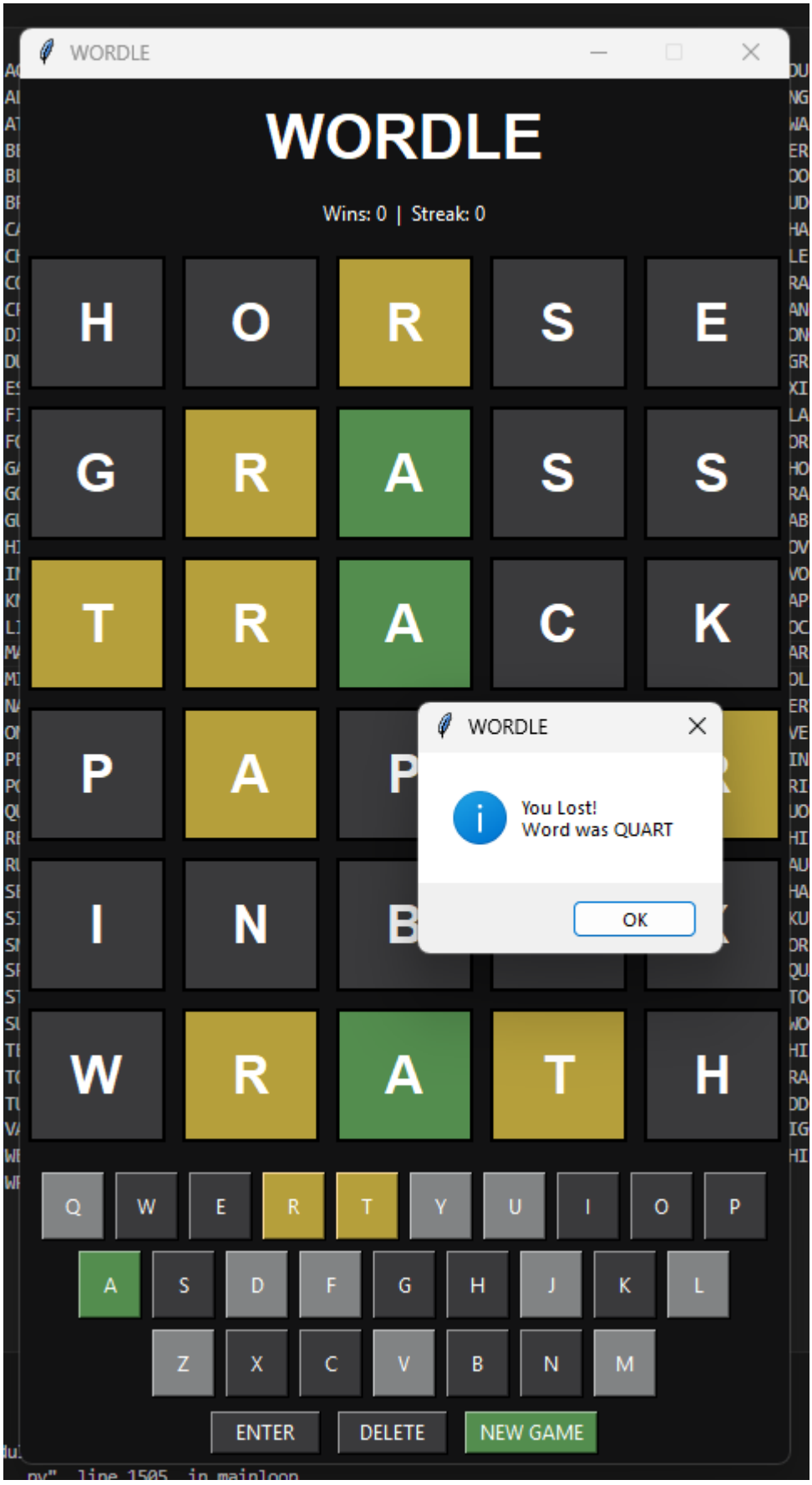
```
            self.board[self.current_row][self.current
_col] = ""
            self.tiles[self.current_row][self.current
_col].config(text="")


    # ======= Wordle logic =======
    def submit_guess(self):
        if self.current_col != 5 or self.game_over:
            return
        guess = "".join(self.board[self.current_row])
        if guess not in self.words:
            messagebox.showwarning("Invalid Word",
"Not in word list")
            return
        colors = ["#3a3a3c"] * 5
        target_copy = list(self.target)
        for i in range(5):  # Green
            if guess[i] == self.target[i]:
                colors[i] = "#538d4e"
                target_copy[i] = None
        for i in range(5):  # Yellow
            if colors[i] == "#3a3a3c" and guess[i] in
target_copy:
                colors[i] = "#b59f3b"
                target_copy[target_copy.index(guess[i
])] = None
        for i in range(5):
            self.tiles[self.current_row][i].config(bg
=colors[i])
```

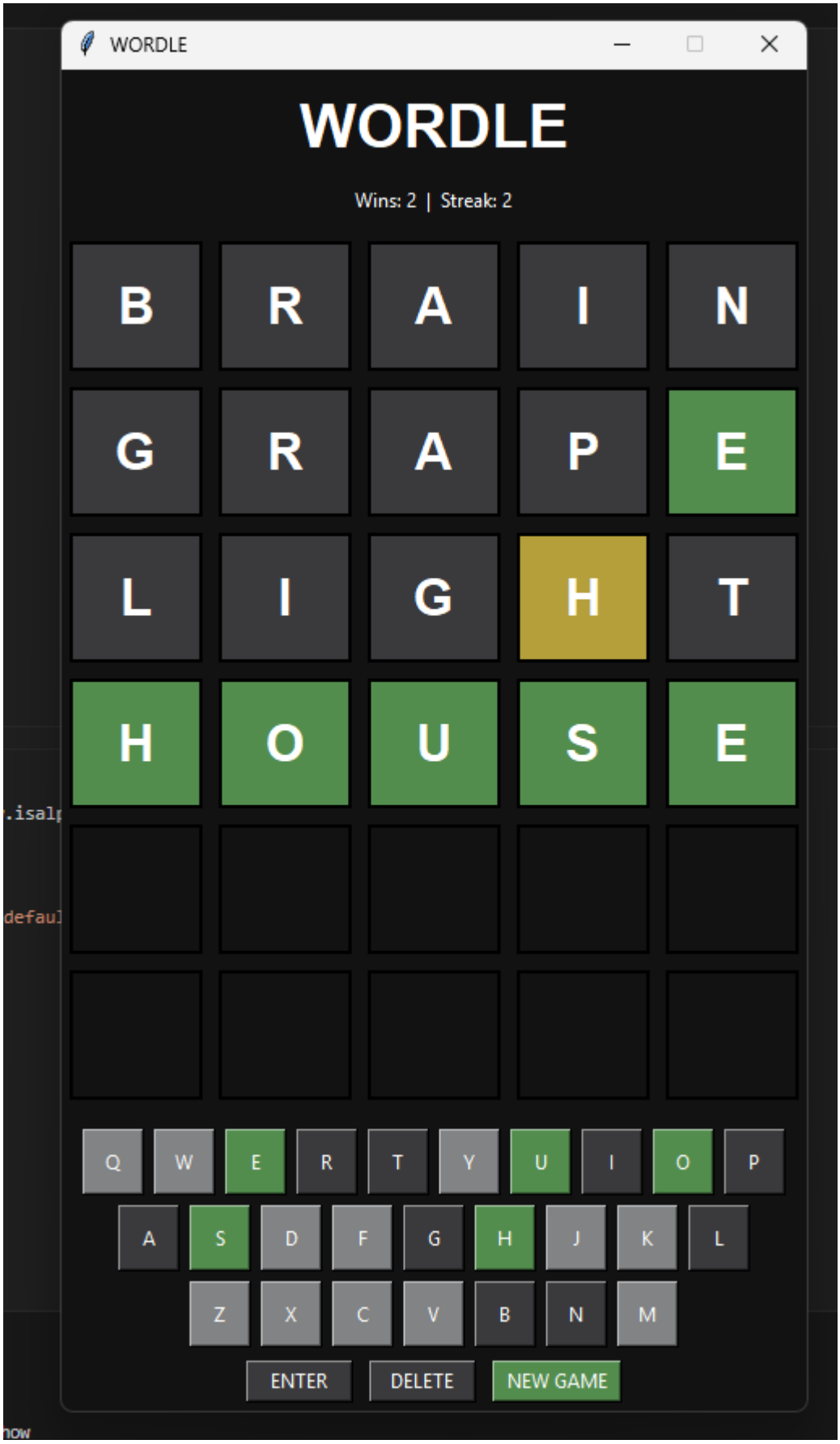
```
            self.update_keyboard_color(guess[i],
colors[i])
        if guess == self.target:
            self.wins += 1
            self.streak += 1
            self.games += 1
            self.update_stats()
            self.game_over = True
            messagebox.showinfo("WORDLE", "You Won!")
            return
        self.current_row += 1
        self.current_col = 0
        if self.current_row == 6:
            self.games += 1
            self.streak = 0
            self.update_stats()
            self.game_over = True
            messagebox.showinfo("WORDLE", f"You
Lost!\nWord was {self.target}")


    def update_keyboard_color(self, letter, color):
        current =
self.keyboard_buttons[letter].cget("bg")
        priority = {"#538d4e": 3, "#b59f3b": 2,
"#3a3a3c": 1, "#818384": 0}
        if priority[color] > priority.get(current,
0):
            self.keyboard_buttons[letter].config(bg=c
olor)
```

```
    def update_stats(self):
        self.stats_label.config(text=f"Wins: {self.wins}  |  Streak: {self.streak}")


# ======= MAIN =======
if __name__ == "__main__":
    root = tk.Tk()
    game = Wordle(root)
    root.mainloop()
```



words.txt


WORDLE
WORDLE
Wins: 0 | Streak: 0
H O R S E
G R A S S
T R A C K
P A P
I N B
W R A T H
WORDLE
You Lost!
Word was QUART
OK
Q W E R T Y U I O P
A S D F G H J K L
Z X C V B N M
ENTER
DELETE
NEW GAME


WORDLE
WORDLE
Wins: 2 | Streak: 2
B R A I N
G R A P E
L I G H T
H O U S E
Q W E R T Y U I O P
A S D F G H J K L
Z X C V B N M
ENTER
DELETE
NEW GAME